

EXPT NO: 1	BUILDING AND TRAINING A SIMPLE ARTIFICIAL NEURAL NETWORK (ANN) FOR REGRESSION AND CLASSIFICATION
DATE: 20 th July,2026	

PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What is an Artificial Neural Network (ANN)?

An Artificial Neural Network (ANN) is a machine learning model inspired by the human brain. It consists of interconnected neurons arranged in input, hidden, and output layers. ANNs learn patterns from data by adjusting weights during training, making them useful for tasks such as prediction, classification, and image recognition.

2. What are the functions of the input, hidden, and output layers in an ANN?

The **input layer** receives the input features from the dataset. The **hidden layer(s)** process the data by applying weights, biases, and activation functions to extract meaningful patterns. The **output layer** produces the final prediction or classification result based on the processed information learned during training.

3. Why is feature scaling important before training an ANN?

Feature scaling ensures that all input features have a similar range of values. This prevents features with larger values from dominating the learning process, improves training stability, speeds up convergence, and helps gradient-based optimization algorithms reach the optimal solution more efficiently and accurately.

4. Differentiate between regression and classification problems with examples.

Regression	Classification
Predicts continuous numerical values.	Predicts discrete categories or labels.
Output is a real number.	Output is a class or category.
Used when the target variable is continuous.	Used when the target variable is categorical.
Example: Predicting house prices or stock prices.	Example: Email spam detection or handwritten digit recognition.
Common evaluation metrics: MAE, MSE, RMSE.	Common evaluation metrics: Accuracy, Precision, Recall, F1-score.

5. What is the role of the activation function in a neural network?

An activation function introduces non-linearity into a neural network, enabling it to learn complex relationships between inputs and outputs. Without activation functions, the network would behave like a simple linear model. Common activation functions include ReLU, Sigmoid, and Tanh, each suited to different learning tasks.

AIM

To understand the implementation of Artificial Neural Networks (ANN) using TensorFlow and evaluate their performance on regression and classification problems.

SOFTWARE REQUIREMENTS

- Python 3.x
- Google Colab / Jupyter Notebook
- TensorFlow 2.x
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

LEARNING OBJECTIVES

After completing this experiment, students will be able to:

1. Understand the architecture of an Artificial Neural Network.
2. Build an ANN for regression problems.
3. Build an ANN for classification problems.
4. Train and evaluate neural network models.
5. Compare different evaluation metrics used for regression and classification.

THEORY

An Artificial Neural Network (ANN) is a machine learning model inspired by the human brain. It consists of interconnected neurons organized into layers.

A simple ANN contains:

- Input Layer
- Hidden Layer(s)
- Output Layer

During training, the ANN learns by updating weights using Backpropagation and an optimization algorithm such as Adam Optimizer.

PART A – ANN FOR REGRESSION
DATASET: California Housing Dataset
SOURCE: Scikit-learn
DESCRIPTION

- Samples: 20,640
- Features: 8
- Target: Median House Value

FEATURES

Feature	Description
MedInc	Median Income
HouseAge	House Age
AveRooms	Average Rooms
AveBedrms	Average Bedrooms
Population	Population
AveOccup	Average Occupancy
Latitude	Latitude
Longitude	Longitude

ALGORITHM

1. Import libraries.
2. Load California Housing dataset.
3. Split into training and testing sets.
4. Standardize the features.
5. Build ANN model.
6. Compile the model.
7. Train the model.
8. Predict house prices.
9. Evaluate using:
 - MSE
 - MAE
 - RMSE

```
import numpy as np
import tensorflow as tf
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

housing = fetch_california_housing()

X = housing.data
y = housing.target

print("Feature Shape:", X.shape)
print("Target Shape:", y.shape)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Data:", X_train.shape)
print("Testing Data :", X_test.shape)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Feature Scaling Completed.")
```

24BAD088_Pradeep_Manikandan_D_DL_24ADI006_Lab_Ex_1.ipynb

Generate

Code

Markdown

Run All

Restart

Clear All Outputs

Python 3.14.0

```

model = Sequential()

model.add(Dense(64, activation='relu', input_shape=(8,)))
model.add(Dense(32, activation='relu'))
model.add(Dense(1))

[22]
Python
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning:
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

model.summary()

[23]
Python
Model: "sequential_1"

Layer (type)                 Output Shape              Param #
dense_3 (Dense)              (None, 64)                576
dense_4 (Dense)              (None, 32)                2,080
dense_5 (Dense)              (None, 1)                 33

```

24BAD088_Pradeep_Manikandan_D_DL_24ADI006_Lab_Ex_1.ipynb

Generate

Code

Markdown

Python 3.14.0

```

Total params: 2,689 (10.50 KB)

Trainable params: 2,689 (10.50 KB)

Non-trainable params: 0 (0.00 B)

model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)

[24]
Python

history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)

[25]
Python
Epoch 1/50

```

24BAD088_Pradeep_Manikandan_D_DL_24ADI006_Lab_Ex_1.ipynb

Generate

Code

Markdown

Run All

Restart

Clear All Outputs

Python 3.14.0

```

Epoch 1/50
413/413 2s 3ms/step - loss: 0.9224 - mae: 0.6591 - val_loss: 0.4872
Epoch 2/50
413/413 3s 3ms/step - loss: 0.4135 - mae: 0.4574 - val_loss: 0.4079
Epoch 3/50
413/413 1s 3ms/step - loss: 0.3714 - mae: 0.4339 - val_loss: 0.4011
Epoch 4/50
413/413 1s 3ms/step - loss: 0.3628 - mae: 0.4250 - val_loss: 0.3782
Epoch 5/50
413/413 1s 3ms/step - loss: 0.3450 - mae: 0.4123 - val_loss: 0.3632
Epoch 6/50
413/413 1s 3ms/step - loss: 0.3290 - mae: 0.4036 - val_loss: 0.3568
Epoch 7/50
413/413 1s 3ms/step - loss: 0.3232 - mae: 0.3981 - val_loss: 0.3825
Epoch 8/50
413/413 1s 3ms/step - loss: 0.3357 - mae: 0.3932 - val_loss: 0.3332
Epoch 9/50
413/413 1s 3ms/step - loss: 0.3081 - mae: 0.3868 - val_loss: 0.3531
Epoch 10/50
413/413 1s 3ms/step - loss: 0.3007 - mae: 0.3812 - val_loss: 0.3240
Epoch 11/50
413/413 1s 4ms/step - loss: 0.2946 - mae: 0.3778 - val_loss: 0.3229
Epoch 12/50
413/413 2s 4ms/step - loss: 0.2953 - mae: 0.3760 - val_loss: 0.3217
Epoch 13/50
...
Epoch 49/50
413/413 1s 2ms/step - loss: 0.2457 - mae: 0.3384 - val_loss: 0.3038
Epoch 50/50

```

24BAD088_Pradeep_Manikandan_D_DL_24ADI006_Lab_Ex_1.ipynb

Generate

Code

Markdown

Python 3.14.0

```

y_pred = model.predict(X_test)
print(y_pred[:10])

[26]
Python
129/129 0s 1ms/step
[[0.362938 ]
 [1.0439924]
 [4.853408 ]
 [2.6174111]
 [2.74687 ]
 [1.6572107]
 [2.442287 ]
 [1.4885968]
 [2.4390044]
 [4.6326194]]

print("Pradeep Manikandan D 24BAD088")
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mse)
print("Mean Squared Error (MSE) :", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)

[27]
Python
Pradeep Manikandan D 24BAD088
Mean Squared Error (MSE) : 0.29650534672704726

```

24ADI006 Deep Learning | Lab EX 1 | 24BAD088 & Pradeep Manikandan D

4

```
print("Pradeep Manikandan D 24BAD088")
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mse)
print("Mean Squared Error (MSE) :", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)
```

```
for i in range(10):
    print(f"Actual: {y_test[i]:.2f} Predicted: {y_pred[i][0]:.2f}")
```

```
print("Mean Squared Error (MSE) :", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)
```

```
for i in range(10):
    print(f"Actual: {y_test[i]:.2f} Predicted: {y_pred[i][0]:.2f}")
```

Mean Squared Error (MSE) : 0.29650534672704726
Mean Absolute Error (MAE): 0.3574587092762561
Root Mean Squared Error (RMSE): 0.5445230451753601

Actual	Predicted
0.48	0.36
0.46	1.04
5.00	4.85
2.19	2.62
2.78	2.75
1.59	1.66
1.98	2.44
1.57	1.49
3.40	2.44
4.47	4.63

PART B – ANN FOR CLASSIFICATION

DATASET: Breast Cancer Wisconsin Dataset

SOURCE: Scikit-learn

DESCRIPTION

- Samples: 569
- Features: 30
- Classes:
 - Malignant
 - Benign

ALGORITHM

1. Import libraries.
2. Load Breast Cancer dataset.
3. Split dataset.
4. Standardize features.
5. Design ANN.
6. Compile model.
7. Train model.
8. Predict labels.
9. Evaluate:
 - Accuracy
 - Precision
 - Recall
 - F1-score
 - Confusion Matrix

```
File Edit Selection View Go Run ... Search
24BAD088_Pradeep_Manikandan_D_DL_24ADI006_Lab_Ex_1.ipynb
Generate + Code + Markdown Run All Restart Python 3.14.0

PART:2 ANN for Classification

import numpy as np
import tensorflow as tf

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.metrics import f1_score, confusion_matrix

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load Breast Cancer Dataset
cancer = load_breast_cancer()
X = cancer.data
y = cancer.target
print("Feature Shape :", X.shape)
print("Target Shape :", y.shape)
print("\nClasses:")
print(cancer.target_names)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
print("Training Samples :", X_train.shape)
print("Testing Samples :", X_test.shape)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
print("Feature Scaling Completed")

Feature Shape : (569, 30)
Target Shape : (569,)
Classes:
['malignant' 'benign']
Training Samples : (455, 30)
Testing Samples : (114, 30)
Feature Scaling Completed
```

```
File Edit Selection View Go Run ... Search
24BAD088_Pradeep_Manikandan_D_DL_24ADI006_Lab_Ex_1.ipynb
Generate + Code + Markdown Run All Restart Clear All Outputs Python 3.14.0

model = Sequential()

model.add(Dense(32, activation='relu', input_shape=(30,)))
model.add(Dense(16, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

model.summary()

Model: "sequential_2"

Layer (type) Output Shape Param #
dense_6 (Dense) (None, 32) 992
dense_7 (Dense) (None, 16) 528
dense_8 (Dense) (None, 1) 17

Total params: 1,537 (6.00 KB)
Trainable params: 1,537 (6.00 KB)
Non-trainable params: 0 (0.00 B)

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)
```

```
Epoch 1/50
12/12 ----- 1s 25ms/step - accuracy: 0.7390 - loss: 0.5972 - val_accuracy: 0.9771
Epoch 2/50
12/12 ----- 0s 8ms/step - accuracy: 0.8736 - loss: 0.4550 - val_accuracy: 0.9771
Epoch 3/50
12/12 ----- 0s 8ms/step - accuracy: 0.9093 - loss: 0.3579 - val_accuracy: 0.9771
Epoch 4/50
12/12 ----- 0s 9ms/step - accuracy: 0.9231 - loss: 0.2900 - val_accuracy: 0.9771
Epoch 5/50
12/12 ----- 0s 9ms/step - accuracy: 0.9258 - loss: 0.2437 - val_accuracy: 0.9771
Epoch 6/50
12/12 ----- 0s 9ms/step - accuracy: 0.9368 - loss: 0.2058 - val_accuracy: 0.9771
Epoch 7/50
12/12 ----- 0s 8ms/step - accuracy: 0.9478 - loss: 0.1789 - val_accuracy: 0.9771
Epoch 8/50
12/12 ----- 0s 8ms/step - accuracy: 0.9478 - loss: 0.1552 - val_accuracy: 0.9771
Epoch 9/50
12/12 ----- 0s 9ms/step - accuracy: 0.9588 - loss: 0.1357 - val_accuracy: 0.9771
Epoch 10/50
12/12 ----- 0s 8ms/step - accuracy: 0.9643 - loss: 0.1205 - val_accuracy: 0.9771
Epoch 11/50
12/12 ----- 0s 8ms/step - accuracy: 0.9670 - loss: 0.1064 - val_accuracy: 0.9771
Epoch 12/50
12/12 ----- 0s 9ms/step - accuracy: 0.9725 - loss: 0.0962 - val_accuracy: 0.9771
Epoch 13/50
12/12 ----- 0s 9ms/step - accuracy: 0.9725 - loss: 0.0962 - val_accuracy: 0.9771
Epoch 49/50
12/12 ----- 0s 14ms/step - accuracy: 0.9945 - loss: 0.0156 - val_accuracy: 0.9771
Epoch 50/50
```

```
y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5).astype(int)
print(y_pred[:10])
```

```
4/4 ----- 0s 34ms/step
[[1]
 [0]
 [0]
 [1]
 [1]
 [0]
 [0]
 [0]
 [1]
 [1]]
```

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
print("\nConfusion Matrix")
print(cm)
```

```
Accuracy : 0.9824561403508771
Precision: 0.9726027397260274
Recall   : 1.0
F1 Score : 0.9861111111111112

Confusion Matrix
[[41  2]
 [ 0 71]]
```

```
for i in range(10):
    print(f"Actual: {y_test[i]} Predicted: {y_pred[i][0]}")
```

```
Actual: 1 Predicted: 1
Actual: 0 Predicted: 0
Actual: 0 Predicted: 0
Actual: 1 Predicted: 1
Actual: 1 Predicted: 1
Actual: 0 Predicted: 0
Actual: 0 Predicted: 0
Actual: 0 Predicted: 0
Actual: 1 Predicted: 1
Actual: 1 Predicted: 1
```

GitHub: [GITHUB REPO](#)

POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. Why is the Adam optimizer commonly used in ANN training?

Adam (Adaptive Moment Estimation) is widely used because it combines the advantages of Momentum and RMSProp. It automatically adjusts the learning rate for each parameter, leading to faster convergence and stable training. Adam works efficiently on large datasets, handles noisy gradients well, and usually requires minimal parameter tuning, making it suitable for most ANN applications.

2. Compare Mean Squared Error (MSE) and Mean Absolute Error (MAE). Which metric is more sensitive to outliers?

Metric	Mean Squared Error (MSE)	Mean Absolute Error (MAE)
Formula	Squares prediction errors	Uses absolute prediction errors
Outlier Sensitivity	Highly sensitive	Less sensitive
Error Penalty	Large errors are penalized more	All errors are treated equally
Use Case	Preferred when large errors should be minimized	Preferred when robustness to outliers is needed

Conclusion: MSE is more sensitive to outliers because squaring the errors gives greater weight to large prediction mistakes.

3. Why is the sigmoid activation function used in binary classification?

The sigmoid activation function is commonly used in binary classification because it converts any input value into a probability between 0 and 1. This output can be interpreted as the likelihood of belonging to the positive class. A threshold, usually 0.5, is then used to classify inputs into one of the two categories.

4. How does increasing the number of hidden layers affect the performance of an ANN?

Increasing the number of hidden layers allows an ANN to learn more complex patterns and relationships in data, improving performance on difficult tasks. However, too many hidden layers can increase computational cost, training time, and the risk of overfitting. Proper regularization and sufficient training data are necessary to achieve better generalization.

5. Compare the evaluation metrics used for regression and classification tasks. Why are different metrics required for these problems?

Aspect	Regression	Classification
Output Type	Continuous values	Discrete class labels
Common Metrics	MSE, MAE, RMSE, R^2 Score	Accuracy, Precision, Recall, F1-Score, ROC-AUC
Goal	Measure prediction error	Measure classification performance

Conclusion: Different metrics are required because regression predicts continuous numerical values, whereas classification predicts categorical labels. Each problem type requires evaluation measures that accurately reflect its objectives and performance.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		